Garantia da Qualidade de Software

Gestão e Qualidade de Software

Professor Daniel Henrique Matos de Paiva

Lista de Exercícios

Esta lista de exercício deve:

- Ser realizada em equipes de até 06 alunos.

- Ser entregue no **prazo** proposto.

- Todos os alunos da equipe devem entregar os exercícios propostos.

- Entregas individuais.

🛠️ Cenário do Projeto: TaskFlow (Gerenciador de Tarefas via Console)

Vamos criar uma aplicação console chamada **TaskFlow** para gerenciar uma lista de tarefas (To-Do List), garantindo alta qualidade de código com testes unitários em **xUnit** e um histórico de commits impecável.

📝 Lista de Exercícios Práticos: Console + xUnit

Crie um repositório no GitHub chamado taskflow-console e execute a sequência abaixo. Para cada passo, utilize o padrão de **Conventional Commits** (tipo(escopo): descrição curta).

1. Inicialização e Estrutura do Projeto

- **Tarefa A:** Crie a solution (dotnet new sln -n TaskFlow), um projeto Console (dotnet new console -n TaskFlow.Console) e adicione-o à solution.
    - *Commit esperado:* chore(core): initialize .NET 10 solution and console project structure
- **Tarefa B:** Crie o projeto de testes unitários com **xUnit** (dotnet new xunit -n TaskFlow.Tests), adicione-o à solution e crie a referência para o projeto Console.
    - *Commit esperado:* chore(test): add xUnit test project and reference to console Project

Garantia da Qualidade de Software

Gestão e Qualidade de Software

Professor Daniel Henrique Matos de Paiva

2. Implementação de Funcionalidades (feat)

- **Tarefa C:** Crie a classe de domínio TaskItem contendo as propriedades Id (Guid ou int), Title, IsCompleted e a lógica básica para alternar o status da tarefa.
    - *Commit esperado:* feat(domain): implement TaskItem entity with completion logic

taskflow-console-net - TaskItem.cs

```
1  public class TaskItem
2  {
3      public Guid Id { get; private set; }
4      public string Title { get; private set; }
5      public bool IsCompleted { get; private set; }
6
7      public TaskItem(string title)
8      {
9          if (string.IsNullOrWhiteSpace(title))
10         {
11             throw new ArgumentException("O título da tarefa não pode ser vazio ou nulo.", nameof(title));
12         }
13
14         Id = Guid.NewGuid();
15         Title = title.Trim();
16         IsCompleted = false;
17     }
18
19     public void Complete()
20     {
21         IsCompleted = true;
22     }
23 }
```

- **Tarefa D:** Crie a classe TaskManager responsável por gerenciar a lista de tarefas em memória (métodos para adicionar, listar e concluir tarefas).
    - *Commit esperado:* feat(services): implement TaskManager logic for in-memory task management

Garantia da Qualidade de Software

Gestão e Qualidade de Software

Professor Daniel Henrique Matos de Paiva

taskflow-console-net - TaskManager.cs

```cs
public class TaskManager
{
    private readonly List<TaskItem> _tasks = new();

    public IReadOnlyCollection<TaskItem> Tasks => _tasks.AsReadOnly();

    public TaskItem AddTask(string title)
    {
        var task = new TaskItem(title);
        _tasks.Add(task);
        return task;
    }

    public TaskItem? GetById(Guid id)
    {
        return _tasks.FirstOrDefault(t => t.Id == id);
    }

    public bool CompleteTask(Guid id)
    {
        var task = GetById(id);
        if (task == null)
        {
            return false;
        }

        task.Complete();
        return true;
    }
}

```

3. Garantindo a Qualidade com Testes (test)

- **Tarefa E:** Crie testes unitários usando **xUnit** para validar se o TaskManager adiciona tarefas corretamente e impede a adição de tarefas com títulos vazios ou nulos.
    - *Commit esperado:* test(unit): add xUnit test suite for TaskManager validation rules

taskflow-console-net - TaskManagerTests.cs

```
public class TaskManagerTests
{
    [Fact]
    public void AddTask_Should_Add_Task_When_Title_Is_Valid()
    {
        // Arrange
        var manager = new TaskManager();
        string title = "Estudar C# e .NET 10";

        // Act
        var task = manager.AddTask(title);

        // Assert
        Assert.NotNull(task);
        Assert.Equal(title, task.Title);
        Assert.False(task.IsCompleted);
        Assert.Single(manager.Tasks);
    }

    [Fact]
    public void AddTask_Should_Throw_Exception_When_Title_Is_Empty()
    {
        // Arrange
        var manager = new TaskManager();

        // Act & Assert
        Assert.Throws<ArgumentException>(() => manager.AddTask("   "));
    }

    [Fact]
    public void GetById_Should_Return_Null_When_Task_Does_Not_Exist()
    {
        // Arrange
        var manager = new TaskManager();

        // Act
        var task = manager.GetById(Guid.NewGuid());

        // Assert
        Assert.Null(task);
    }
}
```

4. Correção de Bugs (fix)

- **Tarefa F:** Durante os testes, notou-se que o método de buscar tarefas por ID lançava uma exceção não tratada (KeyNotFoundException) caso o ID não existisse. Corrija retornando null ou utilizando Nullable<TaskItem>.
    - *Commit esperado:* fix(manager): handle missing task gracefully in search method

5. Refatoração e Melhorias (refactor / docs)

- **Tarefa G:** Refatore a interface do console (Program.cs) para exibir um menu interativo limpo para o usuário interagir com o gerenciador de tarefas.
    - *Commit esperado:* refactor(console): implement interactive menu loop in Program.cs
- **Tarefa H:** Documente no README.md como compilar, rodar a aplicação console e executar os testes unitários.
    - *Commit esperado:* docs(readme): add build, run and test instructions

🚀 **Critérios de Avaliação**

1. **Histórico limpo:** Execute git log --oneline --graph ao terminar para ver o grafo de commits organizado.
2. **Testes Verificando:** Garanta que todos os testes rodem com sucesso executando dotnet test.
3. **Repositório Remoto:** Faça o push de todo o código para o GitHub.